Milestone 4 | Git & GitHub | Interview Questions

Q1. What is the difference between Git and GitHub? Why do people confuse them?

Answer: People confuse them because the names sound similar, but they are completely different things.

	Git	**GitHub**
What is it?	A version control tool that runs on your computer	A website that hosts Git repositories online
Works offline?	✅ Yes — fully local	❌ No — needs internet
Made by	Linus Torvalds, 2005	GitHub Inc. (now owned by Microsoft)
Purpose	Track changes in your code locally	Share code, collaborate with a team, host projects online

Simple analogy:

- Git is like Microsoft Word — the tool you use to write and edit
- GitHub is like Google Drive — the cloud storage where you save and share your documents

```shell
# Git — runs on your machine
git init
git add .
git commit -m "first commit"

# GitHub — you push your local Git work to GitHub
git push origin main
```

> **Interview Tip:** Always clarify this distinction clearly. Many freshers say "I use GitHub" when they mean "I use Git." Interviewers notice this.

Q2. What is a remote repository? How is it different from a local repository?

Answer:

	Local Repository	**Remote Repository**
Where it lives	Your own computer	Server — GitHub, GitLab, Bitbucket
Accessible by team?	❌ No — only you	✅ Yes — anyone with access
Works offline?	✅ Yes	❌ No
Created with	`git init`	Created on GitHub website

```shell
# Local repo - on your machine
cd my-project
git init
git commit -m "add homepage"

# Remote repo - on GitHub
# You link them with:
git remote add origin https://github.com/username/my-project.git

# Then push local → remote
git push origin main
```

> **Interview Tip:** In a real job, your local repo is where you write code. The remote is where the team shares and reviews it. Both are always used together.

Q3. What does `git remote add origin` do? What is "origin"?

Answer: `git remote add origin <url>` connects your local Git repository to a remote repository on GitHub. This connection is saved with a nickname.

`origin` is just a nickname — a short name for the full GitHub URL. It is the standard default name used by convention across the whole industry.

```shell
# Connect local repo to GitHub
git remote add origin https://github.com/rahul-sharma/portfolio.git

# Verify the connection
git remote -v
# Output:
# origin  https://github.com/rahul-sharma/portfolio.git (fetch)
# origin  https://github.com/rahul-sharma/portfolio.git (push)
```

You can technically name it anything, but everyone uses `origin` so the team always knows what it refers to.

> **Common Mistake:** Running `git remote add origin` twice on the same repo causes an error: "remote origin already exists." Fix it with `git remote set-url origin <new-url>`.

Q4. What is `git push`? What does `git push -u origin main` mean?

Answer: `git push` uploads your local commits to the remote repository on GitHub.

```shell
# First push — sets upstream tracking
git push -u origin main

# Every push after that — shorter command works
git push
```

Breaking down `git push -u origin main`:

- `git push` — upload commits
- `-u` — set upstream (remember this remote branch for future pushes)
- `origin` — the remote nickname
- `main` — the branch to push to

Why `-u` on first push? After setting upstream with `-u`, Git remembers the connection. Next time, you just type `git push` without specifying `origin main` every time.

Q5. A fresher at a startup pushed their code and got a "rejected" error. What are the common reasons?

Answer: A push rejection usually means the remote has commits that your local repo does not have. Git refuses to overwrite them.

```shell
# Error message
# ! [rejected] main -> main (fetch first)
# error: failed to push some refs
```

Most common reasons and fixes:

```shell
# Reason 1: Someone else pushed new commits, yours are behind
# Fix: Pull first, then push
git pull origin main
# Resolve any conflicts if they appear
git push origin main

# Reason 2: You and a teammate pushed to the same branch simultaneously
# Fix: Same as above — pull, merge, push

# Reason 3: You tried force pushing without understanding it
# ⚠️ NEVER do this on shared branches
git push --force   # dangerous — overwrites teammates' work
```

> **Interview Tip:** The safe answer is always "pull first, resolve conflicts if any, then push." Force push is only acceptable on your own personal branches.

Q6. What is `git clone`? How is it different from `git init`?

Answer:

	`git init`	`git clone`
Use when	Starting a brand new project	Copying an existing GitHub repo

	`git init`	`git clone`
Downloads existing history?	❌ No	✅ Yes — full commit history
Sets up remote automatically?	❌ No	✅ Yes — `origin` is set up

```shell
# git init – new empty project
mkdir my-project
cd my-project
git init

# git clone – copy a full existing project from GitHub
git clone https://github.com/username/portfolio.git
cd portfolio
# Full project is ready with all history and remote connected
```

Real use: On your first day at a new company in Bengaluru or Hyderabad, you `git clone` the company's project to your laptop. You never run `git init` on a project that already exists.

Q7. What is `git pull`? When should you run it?

Answer: `git pull` downloads the latest commits from the remote repository and merges them into your current local branch. It is a combination of `git fetch` + `git merge`.

```shell
# Get latest changes from GitHub and merge into your branch
git pull origin main
```

When to run it:

- Every morning before starting work — to get your teammates' latest changes
- Before creating a new feature branch — to start from the latest code
- Before pushing — to make sure you are not behind the remote

```shell
# Good daily habit
```

```
git switch main
git pull origin main          # get latest
git switch -c feature-search  # create branch from latest code
```

> **Interview Tip:** Not pulling before starting work is a very common fresher mistake that leads to unnecessary merge conflicts.

Q8. What is the difference between `git pull` and `git fetch`?

Answer:

	`git fetch`	`git pull`
Downloads from remote?	✅ Yes	✅ Yes
Merges into current branch?	❌ No — you decide when to merge	✅ Yes — immediately
Safe to run anytime?	✅ Always safe	⚠️ Can cause conflicts
Use when	You want to review changes before merging	You are ready to update your branch now

```Shell
# fetch — download but don't merge yet
git fetch origin
git log origin/main --oneline   # review what's new
git merge origin/main            # merge when you're ready

# pull — download and merge in one step
git pull origin main
```

Real scenario: Before a meeting at Infosys, you want to see what your team committed overnight without changing your own working files. Use `git fetch`.

Q9. What is a Pull Request (PR)? Why is it important in team development?

Answer: A Pull Request is a request for your teammates to review your code before it gets merged into the main branch. It is a feature of GitHub — not a Git command.

Why it is important:

- Code is reviewed before reaching production — bugs are caught early
- Teammates share knowledge by reading each other's code
- Every merge has a clear record of who approved it and why
- Keeps the main branch stable and clean

Basic PR flow:

```shell
# 1. Create a feature branch
git switch -c feature-job-filter

# 2. Write code, commit it
git add .
git commit -m "Add salary filter to job search"

# 3. Push branch to GitHub
git push origin feature-job-filter

# 4. On GitHub — click "Compare & pull request"
# 5. Add a description of what you built and why
# 6. Request review from a teammate
# 7. Teammate reviews, approves, and merges
```

> **Interview Tip:** Every company — from TCS to Razorpay to Zomato — uses pull requests. Interviewers will ask "have you raised a PR?" Know this flow completely.

Q10. What should a good pull request description include?

Answer: A good PR description helps reviewers understand your changes quickly without reading every line of code.

A good PR includes:

```none
Title: Add salary filter to job search page
```

```
Description:
What changed:
- Added a salary range dropdown to the search bar
- Filter now queries jobs by minimum salary
- Mobile layout adjusted for new filter component

Why:
- Users were requesting this feature since March
- Helps narrow down job results faster

How to test:
- Go to /jobs page
- Open the filter dropdown
- Select "5 LPA - 10 LPA"
- Verify only matching jobs appear

Screenshots: [attach before/after images if UI changed]

Related issue: Closes #42
```

> **Interview Tip:** A well-written PR description is a sign of a professional developer. Many companies ask "how do you write a PR?" in interviews to check communication skills.

Q11. What is a code review? What do reviewers look for?

Answer: A code review is when a teammate reads your code before it is merged — checking for bugs, readability problems, and anything that could be improved.

What reviewers look for:

1. Correctness — Does the code do what it is supposed to do?

JavaScript

```javascript
// ❌ Bug — comparison uses = instead of ===
if (userRole = "admin") { ... }

// ✅ Correct
if (userRole === "admin") { ... }
```

2. Readability — Can others understand it easily?

```JavaScript
// ❌ Bad naming
function fn(x) { return x * 1.18; }

// ✅ Good naming
function addGST(price) { return price * 1.18; }
```

3. No unnecessary code:

```JavaScript
// ❌ Remove console.log before merging
console.log("testing payment flow");
```

4. Proper naming of variables, functions, and CSS classes

5. Any security issues — hardcoded passwords, exposed API keys

> **Interview Tip:** Being able to talk about code review shows maturity. Say: "I follow a checklist — correctness, naming, no debug code, no hardcoded secrets."

Q12. A developer opened a PR but the reviewer asked for changes. What is the correct process?

Answer: When a reviewer requests changes, you make the fixes on the same branch and push again — the PR updates automatically.

```Shell
# You are still on your feature branch
git switch feature-job-filter

# Make the changes the reviewer asked for
# (fix the variable name, remove console.log, etc.)

# Stage and commit the fixes
git add .
git commit -m "Fix: rename variable and remove debug logs per review"
```

```
# Push to the same branch — PR updates automatically on GitHub
git push origin feature-job-filter
```

You do not need to close and reopen the PR. The new commits appear in the same PR and the reviewer can check the changes again.

> **Common Mistake:** Some freshers open a new PR for every review cycle. Always push to the same branch — the existing PR updates on its own.

Q13. What is a merge conflict in a pull request? How do you resolve it?

Answer: A conflict in a PR happens when your branch and the main branch both changed the same line of the same file. GitHub shows a "This branch has conflicts" warning and blocks the merge.

Option 1 — Resolve locally (recommended):

Shell

```
# Step 1: Update your local main
git switch main
git pull origin main

# Step 2: Switch to your feature branch
git switch feature-job-filter

# Step 3: Merge main into your branch
git merge main
# Conflict appears in the file

# Step 4: Open the file, find conflict markers
# <<<<<<< HEAD
# your version
# =======
# teammate's version
# >>>>>>> main

# Step 5: Edit the file to the correct final version, remove markers

# Step 6: Stage and commit
```

```
git add conflicted-file.css
git commit -m "Resolve merge conflict with main"

# Step 7: Push — conflict is resolved in the PR
git push origin feature-job-filter
```

Option 2 — Resolve in GitHub UI: GitHub has a built-in editor for simple conflicts. Click "Resolve conflicts" button on the PR page, edit directly, and mark as resolved.

Q14. What is forking in GitHub? How is it different from cloning?

Answer:

	Forking	**Cloning**
What it does	Creates your own copy of someone else's repo on GitHub	Downloads a repo to your local machine
Where the copy lives	On your GitHub account	On your computer
Can you push changes?	✅ To your fork — then open PR to original	✅ If you have permission
Used for	Contributing to open source or others' projects	Getting a project onto your own machine

```
None
Original repo (rahulcodes/project)
         ↓ fork
Your fork (yourname/project)  ← on GitHub
         ↓ clone
Your machine (local copy)
```

```
Shell
# After forking on GitHub, clone your fork
git clone https://github.com/yourname/project.git
cd project
```

```
# Make changes, commit, push to your fork
git add .
git commit -m "Add dark mode toggle"
git push origin feature-dark-mode

# Then open a PR from your fork to the original repo
```

Q15. What is the open-source contribution workflow? Explain it step by step.

Answer: This is the standard way to contribute to any project on GitHub that you do not own.

Shell

```
# Step 1: Fork the repo on GitHub (click Fork button)
# — creates a copy under your account

# Step 2: Clone your fork
git clone https://github.com/yourname/project.git
cd project

# Step 3: Add the original repo as upstream (so you can sync later)
git remote add upstream https://github.com/originalowner/project.git

# Step 4: Create a branch for your change
git switch -c fix-navbar-mobile

# Step 5: Make your changes and commit
git add .
git commit -m "Fix navbar not showing on mobile screens"

# Step 6: Push to your fork
git push origin fix-navbar-mobile

# Step 7: Go to GitHub, open a PR from your fork to the original repo

# Step 8: Wait for review — respond to comments if any

# Step 9: PR gets merged by the original owner
```

> **Interview Tip:** This workflow is exactly how contributions are made to projects like React, Bootstrap, and thousands of Indian open-source projects. Mention this if asked about open source.

Q16. How do you keep your fork up to date with the original repository?

Answer: When others push new commits to the original repo, your fork falls behind. You sync it using the `upstream` remote.

```shell
# Step 1: Make sure you added upstream (do this once)
git remote add upstream https://github.com/originalowner/project.git

# Step 2: Fetch changes from original repo
git fetch upstream

# Step 3: Switch to your main branch
git switch main

# Step 4: Merge upstream changes into your main
git merge upstream/main

# Step 5: Push updated main to your fork on GitHub
git push origin main
```

> **Interview Tip:** Forgetting to sync your fork is a common open-source contributor mistake. It leads to merge conflicts in your PR. Always sync before starting new work.

Q17. What is GitHub Pages? What kind of websites can you host on it?

Answer: GitHub Pages is a free hosting service from GitHub that lets you publish a website directly from a GitHub repository. The URL looks like `username.github.io/repo-name`.

What you can host:

- ✅ Static websites — HTML, CSS, JavaScript
- ✅ Portfolio websites
- ✅ Project documentation

- ✅ Landing pages

What you cannot host:

- ❌ Backend servers (Node.js, Python, PHP)
- ❌ Databases
- ❌ Server-side code

How to enable GitHub Pages:

```
None
1. Push your project to GitHub
2. Go to Repository → Settings
3. Click "Pages" in the left sidebar
4. Under "Source" – select branch: main, folder: / (root)
5. Click Save
6. Your site goes live at: https://yourname.github.io/repo-name
```

> **Interview Tip:** Every learner should deploy their portfolio on GitHub Pages before interviews. Showing a live URL to an interviewer is much stronger than showing local screenshots.

Q18. A fresher's GitHub Pages site is showing a 404 error after deployment. What are the common reasons?

Answer:

Reason 1 — The main HTML file is not named `index.html`:

```
None
❌ home.html, main.html, portfolio.html
✅ index.html  ← GitHub Pages looks for this by default
```

Reason 2 — Pages not yet enabled:

```
None
Go to Settings → Pages → Check if Source is set to a branch
```

Reason 3 — Wrong branch selected:

```
None
Make sure you selected the correct branch (main or gh-pages) in Pages settings
```

Reason 4 — Files are in a subfolder:

```
None
If index.html is in /src/index.html instead of root, GitHub Pages cannot find it
Move index.html to the root of the repository
```

Reason 5 — Site not yet built — GitHub takes 1-3 minutes:

```
None
Wait a few minutes and refresh. Check Settings → Pages for the green "Your site is
live" message
```

Q19. What is a README file? Why do recruiters care about it?

Answer: A README is a markdown file (`README.md`) at the root of your repository. It is the first thing anyone sees when they visit your project on GitHub.

Why recruiters care:

- It shows you can communicate professionally
- It proves you understand your own project well enough to explain it
- A project without a README looks abandoned or unfinished

A good README includes:

```
None
# Portfolio Website — Priya Sharma

A responsive personal portfolio built with HTML, CSS, and Git.

## Live Demo
[View Portfolio](https://priyasharma.github.io/portfolio)
```

```
## Features
- Fully responsive on mobile, tablet, and desktop
- CSS Grid and Flexbox layout
- Smooth CSS animations on load
- Contact form with basic validation

## Tech Stack
- HTML5
- CSS3 (Flexbox, Grid, Animations)
- Git + GitHub

## How to Run Locally
1. Clone the repo: `git clone https://github.com/priyasharma/portfolio.git`
2. Open `index.html` in your browser

## Screenshots
[Add images here]
```

> **Interview Tip:** Recruiters at companies like Naukri, Internshala, and LinkedIn check GitHub profiles. A clean README with a live link is one of the strongest portfolio signals you can have.

Q20. What is the difference between a public and private repository on GitHub?

Answer:

	Public Repository	**Private Repository**
Who can see it?	Anyone on the internet	Only you and people you invite
Who can clone it?	Anyone	Only collaborators
Cost	Free	Free (with limits)
Good for	Portfolio, open source, learning projects	Company code, client projects, personal work

```
None
When creating a repo on GitHub:
```

```
○ Public  ← recruiters and the world can see this
● Private ← only you and invited collaborators
```

Best practice for learners:

- Portfolio and practice projects → **Public** (recruiters can see them)
- Projects with API keys, passwords, or client data → **Private**

> **Interview Tip:** Never put company code or client project code in a public repo. That is a confidentiality violation and can cost you the job.

Q21. What are GitHub Issues? How are they used in real development teams?

Answer: GitHub Issues is a built-in task and bug tracking system. Think of it like a to-do list for your project that the whole team can see and update.

Common uses:

- Report a bug: "Login button not working on Safari"
- Request a new feature: "Add dark mode"
- Track tasks: "Write tests for payment module"

Basic issue workflow:

```
None
1. Team member or user creates an Issue
2. Issue gets a number – e.g., #42
3. Issue is assigned to a developer
4. Labels are added: bug, enhancement, help wanted
5. Developer works on it, mentions the issue in commit:
   git commit -m "Fix login button on Safari, closes #42"
6. When PR with this commit is merged, Issue #42 is automatically closed
```

> **Interview Tip:** Knowing how Issues work shows you understand professional project management. Mention `closes #issue-number` in commit messages — it auto-closes the issue on merge.

Q22. How do you push a new branch to GitHub so your teammates can see it?

Answer:

```shell
# Step 1: Create and switch to the branch
git switch -c feature-dark-mode

# Step 2: Make your changes and commit
git add .
git commit -m "Add dark mode toggle to navbar"

# Step 3: Push the branch to GitHub
git push origin feature-dark-mode

# Output:
# * [new branch]  feature-dark-mode -> origin/feature-dark-mode
```

After this, teammates can:

```shell
# Pull and switch to your branch
git fetch origin
git switch feature-dark-mode
```

And you can see the branch on GitHub under the "branches" dropdown.

Q23. What does it mean when GitHub says "Your branch is 3 commits behind main"?

Answer: This means 3 new commits were added to `main` on GitHub after you created your branch. Your branch does not have those commits yet.

```
main:        A → B → C → D → E   (3 new commits: C, D, E)
your-branch: A → B → X → Y       (your commits: X, Y)
```

How to fix it:

```shell
# Make sure you are on your feature branch
git switch feature-dashboard

# Fetch latest from GitHub
git fetch origin

# Merge main into your branch
git merge origin/main

# Or use rebase for a cleaner history
git rebase origin/main

# Resolve any conflicts if they appear, then push
git push origin feature-dashboard
```

Keeping your branch up to date reduces the size of conflicts later.

Q24. What is the difference between `git push origin main` and `git push`?

Answer:

```shell
# Explicit — always works, specifies remote and branch
git push origin main

# Short form — only works after upstream is set with -u
git push
```

The short `git push` works only after you have previously run:

```shell
git push -u origin main
# -u sets the upstream tracking so Git remembers
# "this local branch pushes to origin/main"
```

After setting upstream once, `git push` is all you need for that branch.

> **Interview Tip:** Always use `git push -u origin <branch>` for the first push of any new branch. After that, `git push` is enough.

Q25. How would you set up a brand new project from scratch and push it to GitHub?

Answer: This is one of the most common practical interview questions.

```shell
# Step 1: Create project folder and go into it
mkdir portfolio-website
cd portfolio-website

# Step 2: Create your project files
touch index.html style.css
echo "node_modules/" > .gitignore
echo ".env" >> .gitignore

# Step 3: Initialize Git
git init

# Step 4: Stage and make first commit
git add .
git commit -m "Initial project setup"

# Step 5: Create a repo on GitHub website (do not add README on GitHub)

# Step 6: Connect local to GitHub
git remote add origin https://github.com/username/portfolio-website.git

# Step 7: Push to GitHub
git push -u origin main
```

After this, refresh your GitHub page — all files will be visible.

Q26. What is SSH authentication for GitHub? How is it different from HTTPS?

Answer: When pushing to GitHub, you need to prove your identity. There are two methods:

	HTTPS	SSH
How it works	Username + Personal Access Token	Cryptographic key pair
Prompt every time?	Can prompt for token	❌ No — automatic after setup
Setup difficulty	Easy	Medium — one-time setup
Recommended for	Beginners, quick setup	Regular developers — more secure

HTTPS:

```shell
git clone https://github.com/username/project.git
# Asks for username + Personal Access Token on push
```

SSH:

```shell
# One-time setup
ssh-keygen -t ed25519 -C "your@email.com"
# Copy public key to GitHub Settings → SSH Keys

# Then clone using SSH
git clone git@github.com:username/project.git
# No password needed every time
```

> **Interview Tip:** Companies prefer SSH for security. If you have set up SSH keys for GitHub, mention it — it shows you go beyond just the basics.

Q27. What is a Personal Access Token (PAT) in GitHub? Why is it needed?

Answer: GitHub stopped accepting your regular account password for Git operations in August 2021. Instead, you now use a Personal Access Token — a long auto-generated password with specific permissions.

Why it exists:

- More secure than your main account password
- Can be set to expire (e.g., 30 days, 90 days)
- Can be limited to specific actions (read-only, push access only)
- Can be revoked instantly if leaked

How to create one:

```
None
1. GitHub → Settings → Developer Settings
2. Personal Access Tokens → Tokens (classic)
3. Click "Generate new token"
4. Select permissions: repo (full control of repositories)
5. Copy the token — you cannot see it again after this
```

```
Shell
# When Git asks for password during push, use the token instead
Username: your-github-username
Password: ghp_xxxxxxxxxxxxxxxxxxxx  ← paste your token here
```

> **Common Mistake:** Losing the token after closing the browser. Save it in a password manager immediately after generating.

Q28. What should a good GitHub profile look like for a fresher applying to their first job?

Answer: Recruiters from companies like TCS, Wipro, Zoho, and Internshala regularly check candidate GitHub profiles. Here is what makes a strong profile:

Profile level:

- Professional profile picture
- Real name (not a gamer tag)
- Short bio: "Frontend Developer | HTML, CSS, JavaScript | Open to Work"
- Location: your city (Jaipur, Nagpur, Bhopal, etc.)

Repository level — for each project:

```
None
✅ Clear repository name (portfolio-website, not test123)
✅ README with project description and live link
```

```
✅ GitHub Pages URL visible
✅ Clean commit history with meaningful messages
✅ .gitignore present (no node_modules committed)
❌ No empty repositories
❌ No repositories with only one commit named "done"
❌ No sensitive files pushed (passwords, .env files)
```

Activity:

- Regular commits show you are actively practicing
- Green contribution graph (even small practice projects count)

Interview Tip: A portfolio with 3 clean, well-documented projects on GitHub Pages is worth more than 10 messy repos with no READMEs.

Q29. What is `git remote -v`? What does the output tell you?

Answer: `git remote -v` shows all the remote connections your local repo is linked to, and the URL for each.

```Shell
git remote -v

# Output for a regular project:
# origin  https://github.com/rahul/portfolio.git (fetch)
# origin  https://github.com/rahul/portfolio.git (push)

# Output for a forked project with upstream added:
# origin    https://github.com/yourname/project.git (fetch)
# origin    https://github.com/yourname/project.git (push)
# upstream  https://github.com/originalowner/project.git (fetch)
# upstream  https://github.com/originalowner/project.git (push)
```

Each remote shows two URLs — one for fetching (downloading) and one for pushing (uploading). In almost all cases, both URLs are the same.

Q30. A developer cloned a repository but cannot push their changes. What are the common reasons?

Answer:

Reason 1 — You cloned someone else's public repo without forking:

```shell
# You do not have write access to the original repo
# Fix: Fork first, then clone your fork
git clone https://github.com/YOURNAME/project.git  # clone YOUR fork
```

Reason 2 — Not authenticated properly:

```shell
# Error: remote: Invalid username or password
# Fix: Check your Personal Access Token or SSH key setup
git remote set-url origin https://your-token@github.com/username/project.git
```

Reason 3 — Not added as a collaborator:

```
Repo owner must go to Settings → Collaborators → Add your GitHub username
```

Reason 4 — Trying to push to wrong remote:

```shell
git remote -v   # check where push goes
# If URL is wrong:
git remote set-url origin https://github.com/correctname/project.git
```

Q31. What is the difference between forking and cloning? When do you use each?

Answer:

	Fork	Clone
What it creates	A copy on your GitHub account	A copy on your local machine
Needed for contributing to others' projects?	✅ Yes — you fork, then clone	Alone is not enough
Needed to work on your own project?	❌ No	✅ Yes — clone your own repo

When to fork:

- You want to contribute to an open-source project
- You want to use someone's project as a starting point

When to clone:

- Working on your own repository
- Working on a company repository where you are a collaborator

```Shell
# Contributing to someone else's project
1. Fork on GitHub (click Fork button)
2. Clone your fork: git clone https://github.com/YOU/project.git
3. Make changes, push to your fork
4. Open PR to original repo

# Working on your own project
git clone https://github.com/you/your-project.git
```

Q32. How do you add a collaborator to your GitHub repository?

Answer: Collaborators are teammates who can push directly to your repository without needing to fork.

```None
Steps on GitHub:
1. Go to your repository
2. Click "Settings" tab
3. Click "Collaborators" in the left sidebar
```

```
4. Click "Add people"
5. Search by GitHub username or email
6. Click "Add [username] to this repository"
7. They receive an email invitation — they must accept it
```

After accepting, the collaborator can:

```shell
# Clone the repo directly
git clone https://github.com/owner/project.git

# Push branches directly
git push origin feature-payment
```

> **Interview Tip:** In small teams and startups, everyone is added as a collaborator. In large companies, access is managed through organizations with team-level permissions.

Q33. What is `git push origin --delete branch-name`? When would you use it?

Answer: This command deletes a branch from the remote repository on GitHub — after it has been merged and is no longer needed.

```shell
# Delete branch locally
git branch -d feature-login

# Delete the same branch from GitHub (remote)
git push origin --delete feature-login

# Verify it is gone
git branch -r
# origin/feature-login should no longer appear
```

When to use it: After a pull request is merged on GitHub, delete the branch to keep the repository clean. Most GitHub merge screens have a "Delete branch" button that does this automatically.

> **Interview Tip:** Keeping merged branches in remote repos creates clutter. Deleting branches after merge is good team hygiene. Companies include it in their Git workflow guidelines.

Q34. What are GitHub Actions? Briefly, what problem do they solve?

Answer: GitHub Actions is an automation tool built into GitHub. It lets you automatically run tasks when something happens in your repository — like when someone pushes code or opens a pull request.

Common uses:

- Run tests automatically when a PR is opened (so broken code is caught before merging)
- Automatically deploy the website when code is pushed to main
- Check code formatting and flag issues

Simple example — auto-run tests on every push:

```
None
# .github/workflows/test.yml
name: Run Tests

on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - run: npm install
      - run: npm test
```

> **Interview Tip:** You do not need to know GitHub Actions in depth for a fresher role. But knowing it exists and what it does — continuous integration (CI) — is enough to impress interviewers.

Q35. What is the difference between `git merge` in your terminal and merging a PR on GitHub?

Answer: Both do the same thing — combine a branch into another. The difference is where the action happens and who reviews it.

	Terminal `git merge`	**GitHub PR merge**
Review process	❌ No review — immediate	✅ Team reviews code first
Visible to team?	Only if you push afterwards	✅ Full discussion thread visible
Used in real companies?	Only for local merges	✅ Standard for team collaboration
Audit trail?	Only in git log	✅ PR, comments, approvals all saved

```shell
# Local merge — fast, no review
git switch main
git merge feature-login
git push origin main

# GitHub PR merge — reviewed, discussed, documented
# 1. Push branch to GitHub
git push origin feature-login
# 2. Open PR on GitHub
# 3. Team reviews and approves
# 4. Click "Merge pull request" button
```

Q36. How would you explain your Git workflow to an interviewer if they ask "how do you use Git in your project?"

Answer: This is a very common interview question. Here is a clean, confident answer:

"For every new feature or fix, I create a new branch from the updated main branch. I commit my changes regularly with clear commit messages. Once the feature is complete, I push the branch to GitHub and open a pull request describing what I built. I then request a review, make any changes if feedback is given, and the branch is merged into main after approval. After merging, I delete the feature branch to keep the repo clean."

In code terms:

```shell
git switch main
git pull origin main
git switch -c feature-profile-page
# ... work and commit ...
git push origin feature-profile-page
# open PR on GitHub → review → merge
git branch -d feature-profile-page
```

> **Interview Tip:** Practice saying this out loud. Many freshers know Git commands but cannot explain the workflow in words. The ability to explain it clearly is what interviewers remember.

Q37. What should you check before opening a pull request?

Answer: Before opening a PR, run through this checklist:

```shell
# 1. Is your branch up to date with main?
git fetch origin
git merge origin/main
# Resolve any conflicts

# 2. Does your code work? Test it manually.

# 3. Review your own changes first
git diff main..your-branch

# 4. Are commit messages meaningful?
git log --oneline

# 5. Are there any debug logs left?
# Remove all console.log(), print statements

# 6. Is .gitignore set up correctly?
# No node_modules, .env, or build folders committed

# 7. Does the README need to be updated for your change?
```

Only after all these checks, push and open the PR.

> **Interview Tip:** This checklist is the difference between a junior developer who creates problems for reviewers and one who is praised for clean PRs.

Q38. What is the significance of commit history in a GitHub repo? Why do recruiters look at it?

Answer: Commit history is a permanent record of how a project was built — every decision, every fix, every feature, in order.

What good commit history looks like:

```
Shell
git log --oneline

# ✅ Good — tells a story
a3f9b21 Add responsive navbar with Flexbox
7bc1d45 Fix mobile menu not closing after link click
c5d4e3f Add contact form with basic validation
9f8e7d6 Set up project folder structure
```

What bad commit history looks like:

```
Shell
# ❌ Bad — tells nothing
a3f9b21 done
7bc1d45 changes
c5d4e3f fix
9f8e7d6 asdfsdf
```

Why recruiters care:

- It shows how you think and work — methodical vs messy
- It proves you actually built the project yourself (not just downloaded a template)
- It shows you follow professional standards
- Companies like TCS and Infosys check this during background screening of your portfolio

Q39. A teammate deleted the `main` branch on GitHub by mistake. How do you recover it?

Answer: Because Git is distributed, every developer who cloned the repo has a full copy of the history. You can restore it.

```shell
# Step 1: On your local machine, you still have the full history
git branch
# main is still here locally

# Step 2: Push your local main back to GitHub
git push origin main

# If the remote deleted the branch protection
# GitHub also has a "Restore branch" button in the branch list for recently
deleted branches
```

Prevention:

```
On GitHub → Repository Settings → Branches → Add branch protection rule for "main"
Check "Restrict deletions" → No one can delete main
```

> **Interview Tip:** This is a scenario-based question that tests your understanding of how Git works. The key point is: every clone is a full backup. Data is never truly lost unless everyone's copy is gone.

Q40. What is one thing a fresher developer should do on GitHub before going to an interview?

Answer: The single most important thing is to make sure your portfolio project is live on GitHub Pages with a clean README.

Complete pre-interview GitHub checklist:

```
✅ Portfolio project is pushed to GitHub (public repository)
✅ GitHub Pages is enabled — live URL works
✅ README.md has project description, tech stack, and live link
✅ index.html is at the root (not in a subfolder)
```

```
✅ No node_modules, .env, or build files committed
✅ Commit history has meaningful messages (not "done" or "changes")
✅ .gitignore is present
✅ GitHub profile has your real name and a short bio
✅ You can explain every line of your project code
✅ You can walk through the full Git workflow you used
```

What to say in the interview: *"I built my portfolio using HTML, CSS, and Git. The code is on GitHub at [link]. I followed a feature branch workflow — creating branches for each section, committing regularly with clear messages, and it is deployed live at [GitHub Pages link]."*

> **Interview Tip:** Sharing a live GitHub Pages link during an interview — especially in a Zoom screen share — immediately separates you from other freshers who only show screenshots.